Progress report

Bipedal Rescue Robot System

Zhuyun Jin, Debang Ou

Goal & Motivation

Project Goal

Develop a **bipedal rescue robot** capable of navigating cluttered indoor environments and autonomously reaching a target location.

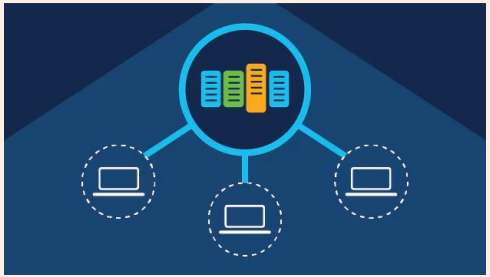
Key Capabilities

- Traverse debris, ramps, and indoor obstacles
- Map unknown environments and localize itself
- Navigate autonomously to a goal location
- Stream telemetry for monitoring and debugging

Application

Our goal is to develop a robot that can safely enter environments that are too dangerous, unstable, or inaccessible for people, while still providing reliable movement, sensing, and visual feedback. In emergency response, it can support firefighters by going first into smoke-filled, low-visibility, or structurally risky spaces, reducing direct human exposure. In hazardous industrial settings, it can inspect areas with toxic air or chemical threats before specialists are sent in. Beyond rescue, we also see strong potential in archaeological exploration, where the robot can move through narrow, fragile, or hard-to-reach spaces and collect visual and spatial data without putting researchers or sensitive sites at unnecessary risk.

System Integration Progress



Remote Robot Control

Robot can now be controlled through a **virtual machine environment**

Commands and telemetry communicate between VM and robot system

Enables remote debugging and development

Perception Capability

Robot sensors can detect **humans and animals**

Uses onboard perception pipeline from camera data

Provides situational awareness for rescue scenarios

Development Environment

Python-based development workflow

ROS2 used for system communication and control

Step 1 — Sensors

Camera, radar, and IMU collect environmental and motion data.

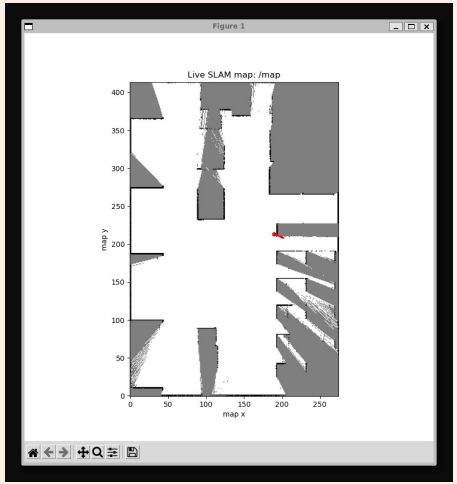
```
                hazard = True
        return hazard, dets

def build_detector() -> Detector:
    if MODEL_PROVIDER == "ultralytics":
        return UltralyticsYOLO(MODEL_CLASSES, MODEL_SCORE_THR)
    elif MODEL_PROVIDER == "onnx":
        return ONNXYOLO(MODEL_PATH, MODEL_CLASSES, MODEL_SCORE_THR, MODEL_NMS_IO
    else:
        return HeuristicBrightness()


def try_enable_camera(client: TronClient):
    """Best-effort camera enable routine for firmware variants."""
    if not CAMERA_ENABLE_ON_START:
        return
    print("[camera] trying enable commands:", ", ".join(CAMERA_ENABLE_COMMANDS))
    for cmd in CAMERA_ENABLE_COMMANDS:
        try:
            client.send(cmd)
        except Exception as e:
            print(f"[camera] send failed for {cmd}: {e}")
            continue

        time.sleep(max(0.05, CAMERA_ENABLE_WAIT))
        st = client.last_status or {}
        cam = str(st.get("camera", "UNKNOWN")).upper()
        mode = st.get("status")
        print(f"[camera] after {cmd}: camera={cam} mode={mode}")
        if cam == "OK":
            print(f"[camera] camera enabled by {cmd}")
            return

    print("[camera] camera still not OK after all commands.")
    print("[camera] If needed, override CAMERA_ENABLE_COMMANDS with your firmwar
```

Step 2 — Perception & Mapping

Visual-inertial odometry

ORB-SLAM3 for mapping and localization

Human / animal detection from camera input

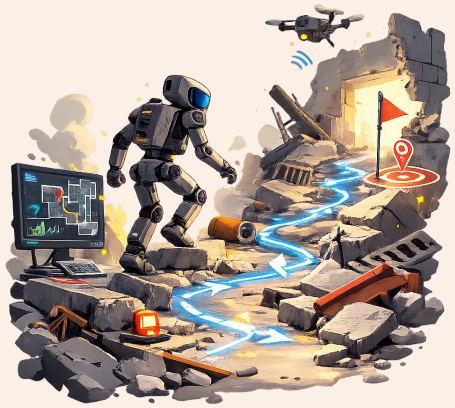


Step 3 — Planning & Control

ROS2 Nav2 for navigation planning

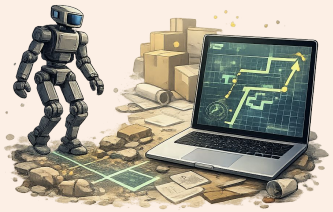
Reinforcement learning policy for locomotion

Safety layer for fall detection and recovery



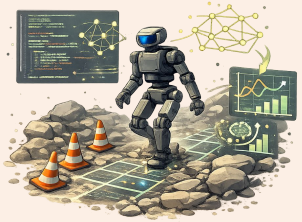
Upcoming Development Milestones

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam convallis, dolor non ornare rutrum, leo metus varius orci, ac sodales enim nibh in nibh.



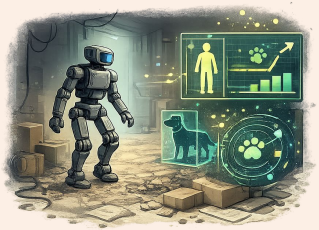
Locomotion Training

Train reinforcement learning policies for stable walking and obstacle traversal.



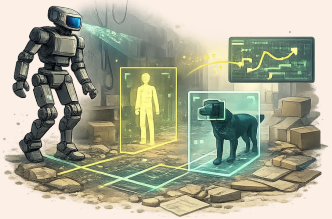
Navigation Integration

Connect SLAM and navigation systems to enable **goal-directed autonomous movement**.



Detection Integration

Use human/animal detection results to improve navigation decisions and safety.



Final Demo

Robot receives a goal → maps environment → plans path → traverses obstacles → reaches target.